

AIops — Module 1

Raghav Iyengar (DA24B020) | Partner (Q4): Dhanush (DA24B019) | <https://github.com/DA24B020/aiops-as1>

1. Technical Debt

Reported symptom	Debt category
(a) Rounding change to <i>estimated delivery time</i> degraded the unrelated <i>favorite restaurants</i> feature	Entanglement (CACE)
(b) Marketing dashboard silently reading the model's raw output table	Undeclared Consumer
(c) Training pipeline is 14 undocumented shell scripts, no orchestration	Configuration and Glue Code Debt

Proposed mitigation

The fix would be to replace the tangled shell scripts with a single DVC pipeline that declares each step, keeping all settings in a configuration file and log the runs using a package like MLflow. This will ensure that each step has clear control on what it reads and writes. This will ensure that the order is documented and only the steps whose inputs are changed will be re-run and the code/data change can be easily tracked so that changes can be viewed/undone.

2. MLflow Experiments

Analysis

Seven MLP configurations were tracked in MLflow, varying learning rate (1e-4, 1e-3, 1e-2) and hidden-layer width/depth. Learning rate dominated the outcome. All three low-LR runs (exp1, exp3, exp5) began near 0.44–0.71 validation accuracy and were still improving at step 20, ending at 0.9122, 0.9184 and 0.9331 respectively — clearly under-trained rather than converged, with validation loss still falling steeply. The high-LR runs (exp2, exp4, exp6) started above 0.89, reached their plateau within two or three epochs, and settled at 0.9483, 0.9616 and 0.9547, with loss flattening near zero and showing minor step-to-step noise thereafter.

Width helped, but much less than learning rate: at fixed LR, going from 50 to 200 units gained roughly 2 points at 1e-4 and under 1 point at 1e-2, with 100 units (exp4) actually the best overall at 0.9616. The deeper (100, 50) network at 1e-3 reached 0.9572, matching the wide shallow models without beating them.

Best configuration: exp4-medium-highlr, lr=0.01, hidden=(100,), val_acc=0.9616. The low-LR runs would likely need substantially more epochs to be judged fairly.

```
mlflow.log_param("hidden_layer_sizes", str(hidden_layer_sizes)) # 3+ hyperparameters
mlflow.log_param("learning_rate", learning_rate)
mlflow.log_param("alpha", alpha); mlflow.log_param("max_epochs", max_epochs)
for epoch, (train_loss, val_acc) in enumerate(zip(train_losses, val_accs)):
    mlflow.log_metric("train_loss", train_loss, step=epoch) # step -> time series
    mlflow.log_metric("val_accuracy", val_acc, step=epoch)
mlflow.log_metric("test_accuracy", test_acc)
```

	Run Name	Created	Models	Metrics				Parameters		
				test_accuracy	train_loss	val_accuracy	alpha	hidden_layer_1	hidden_layer_2	learning_rate
☐	exp1-small-lowlr	6 minutes ago	mlflow	0.141666...	2.190320...	0.170138...	0.0001	(5)		0.0001
☐	exp2-small-highlr	6 minutes ago	mlflow	0.944444...	0.089037...	0.972222...	0.0001	(5)		0.01
☐	exp3-medium-lowlr	6 minutes ago	mlflow	0.458333...	2.053487...	0.354166...	0.0001	(100)		0.0001
☐	exp4-medium-highlr	6 minutes ago	mlflow	0.961111...	0.063445...	0.965277...	0.0001	(100)		0.01
☐	exp5-large-lowlr	6 minutes ago	mlflow	0.650444...	1.948782...	0.642861...	0.0001	(200)		0.0001
☐	exp6-large-highlr	6 minutes ago	mlflow	0.961111...	0.063445...	0.965277...	0.0001	(200)		0.01
☐	exp7-deep-medlr	6 minutes ago	mlflow	0.936111...	0.223708...	0.923611...	0.0001	(100, 50)		0.001

3. DVC Versioning & Rollback

Remote: s3://aiops-da24b019-da24b020/q3/dvcstore (ap-south-1). Full transcript: q3/evidence/q3_transcript.txt.

Version	Tag	Rows (lines)	.dvc md5
v1	v1	1800 (1801)	1cb5fc7e34e8c20bd951d18577866870
v2	v2	2800 (2801)	56da1dbfeb097b12d441e0777ae1c5ac

Rollback. After `git checkout v1` the workspace file still had **2801** lines: Git restored the 5-line `.dvc` pointer but not the data, which it never stored. `dvc checkout` then read the cached content matching that pointer and materialised the **1801**-line v1 file, with `md5sum` equal to the hash recorded in the pointer. Neither command is sufficient alone — that two-layer split is what makes rollback reliable.

4. Reproducibility Drill

Shared repo: <https://github.com/DA24B020/aiops-m1-da24b019-da24b020> (commits prefixed [A]/[B] for attribution). Transcript: q4/evidence/q4_transcript.txt.

Partner A committed training code and the `.dvc` pointer in one commit (ac32533) with a clean tree, then trained: run 1bfc37a0, seed 42, `git_dirty=false`, `final_val_accuracy = 1.0000`; registered as `m1-capstone-model v3` and transitioned to **Staging**. **Partner B** cloned, checked out ac32533, ran `dvc pull && dvc checkout` (data md5 matched, so both trained on byte-identical input), built the environment and reran: run 47ff794f, 1.0000. **Verdict: MATCH**, $|\Delta| = 0.0000$ against a tolerance of ± 0.005 declared before comparing, logged to Partner B's run as the `repro_note` tag.

Caveat. The metric reproduced but the *environment* did not: `pip install -r requirements.txt` failed on Partner B's machine (`pip freeze` over-pinned `botocore` against `aiobotocore`'s range), and he ran Python 3.14 against my 3.13, which a requirements file cannot pin. The match reflects the robustness of a fixed-seed model on a small dataset rather than a fully reproduced environment; a lockfile plus a pinned interpreter would close the gap.